

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Fundamentals of Data Science

COURSE CODE: DSA0401

1. Online Food Delivery Data Analysis

An online food delivery company wants to analyze order data to improve delivery performance and customer satisfaction. Explain how:

A. Pandas is used to read, clean, and filter order data

B. Data preprocessing is applied to handle missing delivery time, duplicate orders, and incorrect prices

C. Matplotlib or Seaborn is used to visualize delivery delays, top restaurants, and customer Ratings

Sol:

Aim

To analyze an online food delivery dataset using Python by performing data cleaning, preprocessing, and visualization techniques.

Problem Statement

Online food delivery platforms generate large amounts of customer and restaurant data. The dataset may contain missing values, duplicate records, and invalid entries that can affect analysis. The objective is to clean the dataset and visualize important information such as delivery time, restaurant popularity, and customer ratings.

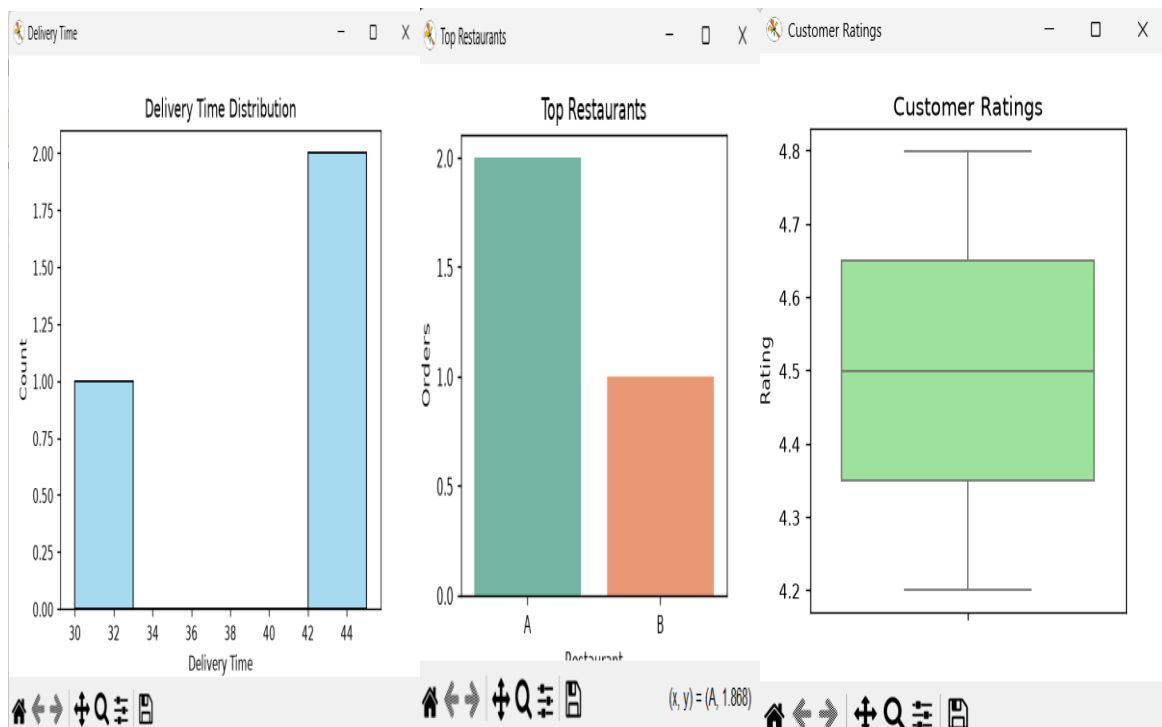
Solution

- Created a DataFrame containing restaurant details, delivery time, price, and customer ratings.
- Replaced missing delivery time values using the mean.
- Removed duplicate records.
- Filtered records with invalid (negative) prices.
- Displayed the cleaned dataset.
- Generated a Histogram for delivery time distribution.
- Generated a Count Plot for restaurant-wise orders.
- Generated a Box Plot for customer ratings.

Code:

```
A13-1.py > ...
1  import pandas as pd
2  import matplotlib.pyplot as plt
3  import seaborn as sns
4
5  data = {
6      "Restaurant": ["A", "B", "A", "C", "B", "A"],
7      "Delivery_Time": [30, 45, None, 60, 45, 30],
8      "Price": [250, 300, 200, -50, 300, 250],
9      "Rating": [4.5, 4.2, 4.8, 3.9, 4.2, 4.5]
10 }
11
12 df = pd.DataFrame(data)
13
14 print("Original Data")
15 print(df)
16
17 df["Delivery_Time"] = df["Delivery_Time"].fillna(df["Delivery_Time"].mean())
18 df = df.drop_duplicates()
19 df = df[df["Price"] > 0]
20
21 print("\nProcessed Data")
22 print(df)
23
24 plt.figure("Delivery Time", figsize=(6,4))
25 sns.histplot(df["Delivery_Time"], bins=5, color="skyblue")
26 plt.title("Delivery Time Distribution")
27 plt.xlabel("Delivery Time")
28 plt.ylabel("Count")
29
30 plt.figure("Top Restaurants", figsize=(6,4))
31 sns.countplot(x="Restaurant", data=df, palette="Set2")
32 plt.title("Top Restaurants")
33 plt.xlabel("Restaurant")
34 plt.ylabel("Orders")
35
36 plt.figure("Customer Ratings", figsize=(6,4))
37 sns.boxplot(y="Rating", data=df, color="lightgreen")
38 plt.title("Customer Ratings")
39 plt.ylabel("Rating")
40
41 plt.show(block=False)
42 plt.pause(0.1)
43
44 input("Press Enter to close all graphs...")
45 plt.close('all')
```

Output:



2. Bank Loan Approval Analysis

A bank wants to study customer loan applications and identify patterns related to loan approval and rejection. Explain how:

A. Pandas DataFrame operations are used to select income, loan amount, credit score, and approval status

B. Preprocessing techniques are used to handle missing income values, encode gender/employment type, and normalize loan amount

C. Visualization is used to compare approved and rejected applications based on income, credit score, and loan amount

Sol:

Aim

To preprocess and analyze a bank loan approval dataset using Python libraries and visualize the relationship between customer information and loan approval.

Problem Statement

Loan approval depends on several customer attributes such as income, employment type, and credit score. Missing values and categorical data must be processed before analysis. The objective is to preprocess the dataset and visualize the factors influencing loan approval.

Solution

- Created a DataFrame containing customer income, loan amount, credit score, gender, employment type, and approval status.
- Filled missing income values using the mean.
- Encoded categorical variables using Label Encoding.
- Normalized loan amounts using Min-Max Scaling.
- Displayed the processed dataset.
- Generated a Scatter Plot showing Income vs Credit Score.
- Generated a Box Plot comparing loan amounts based on approval status.

Code:

```
AT3-2.py > ...
26 scaler = MinMaxScaler()
27 df["Loan"] = scaler.fit_transform(df[["Loan"]])
28
29 print("\nProcessed Data")
30 print(df)
31
32 plt.figure("Income vs Credit Score", figsize=(6, 4))
33 sns.scatterplot(x="Income", y="CreditScore", hue="Approval", data=df, s=100)
34 plt.title("Income vs Credit Score")
35 plt.xlabel("Income")
36 plt.ylabel("Credit Score")
37
38 plt.figure("Loan Approval Comparison", figsize=(6, 4))
39 sns.boxplot(x="Approval", y="Loan", data=df, palette="Set2")
40 plt.title("Loan Amount by Approval")
41 plt.xlabel("Approval")
42 plt.ylabel("Normalized Loan Amount")
43
44 plt.show(block=False)
45 plt.pause(0.1)
46
47 input("Press Enter to close all graphs...")
48 plt.close("all")
```

Output:



3. Smart Energy Consumption Monitoring

A smart home system collects daily electricity consumption data from different appliances such as AC, fan, light, and refrigerator. Explain how:

A. Pandas is used to group electricity usage by appliance and date

B. Data preprocessing is used to remove abnormal readings, fill missing values, and calculate total power consumption

C. Line charts

Sol:

Aim

To analyze smart energy consumption data by cleaning the dataset and visualizing power usage using Python.

Problem Statement

Energy monitoring systems collect appliance-wise power consumption data. Missing values and abnormal readings can reduce analysis accuracy. The objective is to clean the dataset, calculate total power consumption, and visualize daily and appliance-wise energy usage.

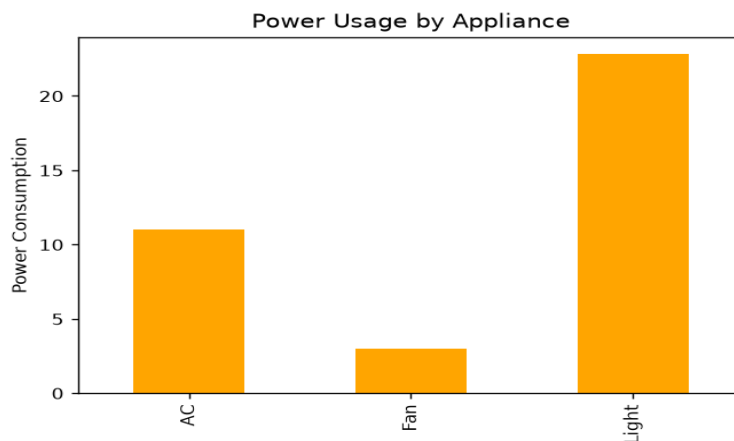
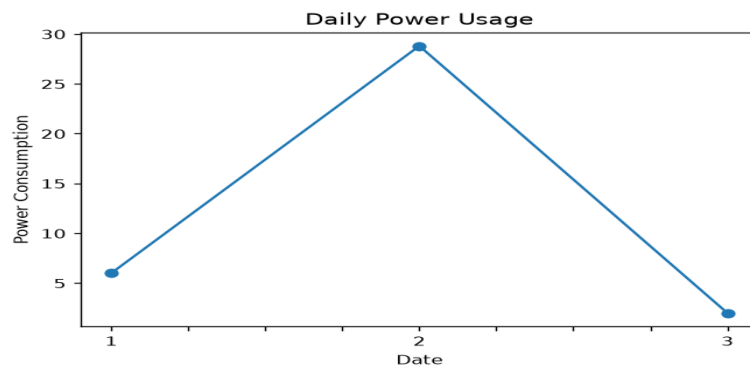
Solution

- Created a DataFrame containing appliance names, dates, and power consumption values.
- Displayed appliance-wise power consumption using the `groupby()` function.
- Filled missing power values using the mean.
- Removed abnormal power values greater than 50.
- Calculated total power consumption.
- Displayed the cleaned dataset.
- Generated a Line Chart showing daily power consumption.
- Generated a Bar Chart showing appliance-wise power consumption.

Code:

```
AT3-3.py > ...
17
18 print("\nProcessed Data")
19 print(df)
20
21 print("\nTotal Power Consumption =", df["Power"].sum())
22
23 plt.figure("Daily Power Usage", figsize=(6, 4))
24 df.groupby("Date")["Power"].sum().plot(kind="line", marker="o")
25 plt.title("Daily Power Usage")
26 plt.xlabel("Date")
27 plt.ylabel("Power Consumption")
28
29 plt.figure("Appliance Power Usage", figsize=(6, 4))
30 df.groupby("Appliance")["Power"].sum().plot(kind="bar", color="orange")
31 plt.title("Power Usage by Appliance")
32 plt.xlabel("Appliance")
33 plt.ylabel("Power Consumption")
34
35 plt.show(block=False)
36 plt.pause(0.1)
37
38 input("Press Enter to close all graphs...")
39 plt.close("all")
```

Output:



4. Online Learning Platform Performance Analysis

An online learning platform wants to analyze student activity such as login time, video completion, quiz score, and assignment submission. Explain how:

A. Pandas is used to sort, filter, and rank students based on quiz scores and completion percentage

B. Preprocessing is applied to handle missing quiz scores, convert date formats, and create

new columns such as performance category

C. Seaborn and Matplotlib are used to visualize student engagement, score distribution, and course completion status

Sol:

Aim

To analyze student performance data using Python by performing preprocessing and visualizing quiz scores and course completion.

Problem Statement

Online learning platforms store student information such as quiz scores, login dates, and course completion status. Missing values and inconsistent data should be processed before analysis. The objective is to clean the dataset, classify student performance, and visualize learning outcomes.

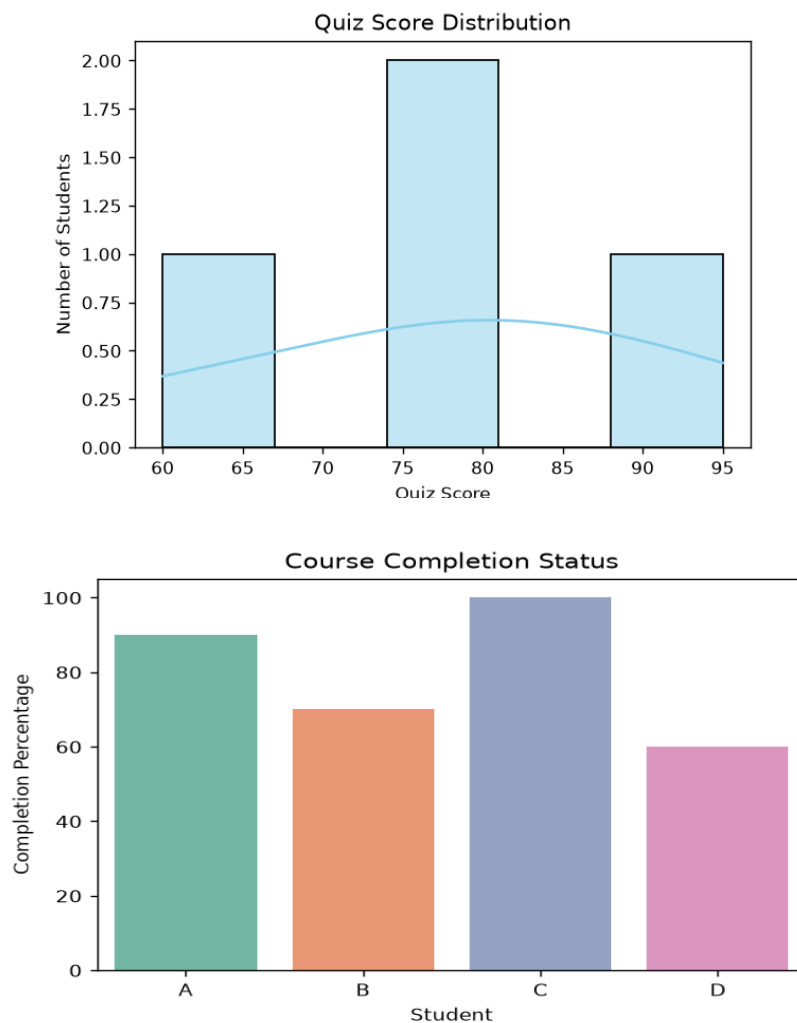
Solution

- Created a DataFrame containing student details, quiz scores, completion percentage, and login dates.
- Filled missing quiz scores using the mean.
- Converted login dates into DateTime format.
- Classified students into Good and Average performance categories.
- Sorted students based on quiz scores.
- Displayed the processed dataset.
- Generated a Histogram for quiz score distribution.
- Generated a Bar Chart for course completion percentage.

Code:

```
AT3-4.py > ...
24 print("\nSorted Students")
25 print(df.sort_values("Quiz", ascending=False))
26
27 print("\nProcessed Data")
28 print(df)
29
30 plt.figure("Quiz Score Distribution", figsize=(6,4))
31 sns.histplot(df["Quiz"], bins=5, kde=True, color="skyblue")
32 plt.title("Quiz Score Distribution")
33 plt.xlabel("Quiz Score")
34 plt.ylabel("Number of Students")
35
36 plt.figure("Course Completion", figsize=(6,4))
37 sns.barplot(x="Student", y="Completion", data=df, palette="Set2")
38 plt.title("Course Completion Status")
39 plt.xlabel("Student")
40 plt.ylabel("Completion Percentage")
41
42 plt.show(block=False)
43 plt.pause(0.1)
44
45 input("Press Enter to close all graphs...")
46 plt.close("all")
```

Output:



5. Public Transport Passenger Data Analysis

A city transport department wants to analyze bus passenger data to improve route planning

and reduce overcrowding. Explain how:

A. Pandas is used to analyze passenger count by route, stop, and time slot

B. Data preprocessing is used to clean invalid route numbers, handle missing passenger counts, and remove duplicate records

C. Visualization techniques are used to show peak travel hours, busiest routes, and passenger flow patterns

Sol:

Aim

To analyze public transport passenger data using Python by cleaning the dataset and visualizing passenger traffic patterns.

Problem Statement

Public transport systems collect passenger data for different routes and travel times. Missing values, duplicate records, and invalid route numbers can affect analysis. The objective is to preprocess the dataset and visualize peak travel hours and the busiest routes.

Solution

- Created a DataFrame containing route numbers, passenger counts, and travel times.
- Displayed passenger count grouped by route.
- Filled missing passenger values using the mean.
- Removed invalid route numbers containing non-numeric values.
- Removed duplicate records.
- Displayed the cleaned dataset.
- Generated a Count Plot showing peak travel hours.
- Generated a Bar Chart showing passenger count for each route.

Code:

```
AT3-5.py > ...
1  import pandas as pd
2  import matplotlib.pyplot as plt
3  import seaborn as sns
4
5  data = {
6      "Route": ["101", "102", "101", "103", "ABC", "102"],
7      "Passengers": [50, 70, None, 90, 40, 70],
8      "Time": ["8AM", "9AM", "8AM", "6PM", "9AM", "9AM"]
9  }
10
11  df = pd.DataFrame(data)
12
13  print("Original Data")
14  print(df)
15
16  print("\nPassengers by Route")
17  print(df.groupby("Route")["Passengers"].sum())
18
19  df["Passengers"] = df["Passengers"].fillna(df["Passengers"].mean())
20  df = df[df["Route"].str.isnumeric()]
21  df = df.drop_duplicates()
22
23  print("\nProcessed Data")
24  print(df)
25
26  plt.figure("Peak Hours", figsize=(6, 4))
27  sns.countplot(x="Time", data=df, palette="Set2")
28  plt.title("Peak Hours")
29  plt.xlabel("Time")
30  plt.ylabel("Number of Trips")
31
32  plt.figure("Busiest Routes", figsize=(6, 4))
33  df.groupby("Route")["Passengers"].sum().plot(kind="bar", color="skyblue")
34  plt.title("Busiest Routes")
35  plt.xlabel("Route")
36  plt.ylabel("Total Passengers")
37
38  plt.show(block=False)
39  plt.pause(0.1)
40
41  input("Press Enter to close all graphs...")
42  plt.close("all")
```

Output:

